🚀 OBJECT ORIENTED SYSTEMS (OOS) — QUESTION 2 COMPLETE SOLUTION

📄 QUESTION 2(a)

❓ **Distinguish the functionalities between throw and throws keyword with suitable code.**
❓ **Provide a scenario where both of them can be used in a combined way.**

✅ Answer

Exception handling is one of the most powerful features of Java 🚀. It allows programmers to manage runtime abnormalities gracefully instead of terminating the program abruptly. Java provides two important keywords related to exceptions:

- 🔥 `throw`
- 🔥 `throws`

Although they look similar, their functionalities are completely different.

🌟 What is `throw`?

The `throw` keyword is used to explicitly generate an exception object manually. It is generally used inside:

- Methods
- Constructors
- Blocks

...to signal an abnormal condition.

Syntax:

```
throw new ExceptionType();
```

Example:

```
throw new ArithmeticException("Division by zero");
```

🌟 What is `throws`?

The `throws` keyword is used in the method declaration to indicate that the method may generate one or more exceptions. It informs the caller that exception handling responsibility is transferred upward.

Syntax:

```
void show() throws IOException
```

🌟 Difference Between `throw` and `throws`

Feature	`throw`	`throws`
Purpose	Explicitly generates exception	Declares possible exception
Used Inside	Method body	Method declaration
Followed By	Exception object	Exception class name
Number of Exceptions	One object at a time	Multiple exceptions possible
Responsibility	Creates exception	Passes responsibility upward

✅ Example Using `throw`

```
class ThrowDemo {
    public static void main(String[] args) {
        int age = 15;
        if(age < 18) {
            throw new ArithmeticException("Not eligible for voting");
        }
        System.out.println("Eligible");
    }
}
```

Output:

```
Exception in thread "main"
java.lang.ArithmeticException: Not eligible for voting
```

✅ Example Using `throws`

```
import java.io.*;

class ThrowsDemo {
    void readFile() throws IOException {
        FileReader fr = new FileReader("abc.txt");
        System.out.println("File Opened");
    }

    public static void main(String[] args) {
        ThrowsDemo obj = new ThrowsDemo();
        try {
```

```
            obj.readFile();
        } catch(IOException e) {
            System.out.println("Exception Handled");
        }
    }
}
```

Output:

```
Exception Handled
```

🌟 Combined Use of `throw` and `throws`

Both keywords are often used together in real-world programs. The method:

1. Creates the exception using `throw`
2. Passes responsibility using `throws`

✅ Combined Example

```
class CombinedDemo {
    void checkAge(int age) throws ArithmeticException {
        if(age < 18) {
            throw new ArithmeticException("Underage Person");
        } else {
            System.out.println("Eligible for Voting");
        }
    }

    public static void main(String[] args) {
        CombinedDemo obj = new CombinedDemo();
        try {
            obj.checkAge(15);
        } catch(ArithmeticException e) {
            System.out.println("Exception Caught: " + e.getMessage());
        }
    }
}
```

Output:

```
Exception Caught: Underage Person
```

💡 Deep Explanation

Here:

- ✅ `throws` declares that the method may generate an exception.
- ✅ `throw` actually creates the exception object.

Thus both work together beautifully in exception propagation 🚀.

🎯 Conclusion

The `throw` keyword is used to explicitly generate exceptions, whereas `throws` is used to declare exception responsibility. Together they provide powerful exception propagation and handling mechanisms in Java 🧠✨.

📝 QUESTION 2(b)

❓ **"Any member method in a Base class while overridden in Child class should be given more strict access specifier."**
❓ **Justify the validity of the statement with suitable code.**

❌ The Statement is FALSE

✅ Explanation

When method overriding occurs in Java, the access specifier of the overridden method in the child class 🚫 **CANNOT be more restrictive** than the access specifier of the parent class method. This is one of the most important rules of method overriding ⚠️.

🌟 Why This Restriction Exists?

Suppose a method is accessible publicly in the parent class. If the child class reduces visibility, then polymorphism breaks.

- **Example:** A parent reference should always access overridden child methods safely. Thus Java prevents reducing accessibility.

🌟 Valid Access Expansion (Access can become same or less restrictive)

Parent	Child	Status
`protected`	`public`	✅ Valid
`default`	`protected`	✅ Valid
`protected`	`protected`	✅ Valid

🌟 Invalid Access Reduction

Parent	Child	Status
`public`	`protected`	❌ Invalid
`protected`	`private`	❌ Invalid

✅ Invalid Program

```
class Base {
    public void show() {
        System.out.println("Base show()");
    }
}

class Child extends Base {
    // INVALID OVERRIDING
    protected void show() {
        System.out.println("Child show()");
    }
}
```

⚠️ Compiler Error:

```
Cannot reduce the visibility of inherited method
```

✅ Correct Program

```
class Base {
    protected void show() {
        System.out.println("Base show()");
    }
}

class Child extends Base {
    // VALID
    public void show() {
        System.out.println("Child show()");
    }
}

class Demo {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
    }
}
```

Output:

```
Child show()
```

🌟 Important Rule of Overriding

During overriding:

- ✅ Access can remain the same
- ✅ Access can become less restrictive
- ❌ Access cannot become more restrictive

🌟 Real-World Intuition

Imagine a parent company gives public access to a service. A branch office cannot suddenly hide that service from users. Similarly, child classes cannot reduce inherited accessibility 🚀.

🎯 Conclusion

The statement is invalid because Java does not allow stricter access specifiers during method overriding. The child method must maintain or increase accessibility to preserve polymorphism and inheritance consistency 🧠✨.

📃 QUESTION 2(c)

❓ **Fill up the blanks with suitable phrases.**

✅ Answers

1. A try block may be followed by: ✅ **catch block(s)**
2. A try-catch block may be followed by: ✅ **finally block**
3. ✅ **final** member method of a Base class is restricted to be overridden in its Child class.
4. A String is ✅ **immutable** datatype while a StringBuffer is ✅ **mutable** datatype.
5. Method overriding is also known as ✅ **dynamic** binding.
6. An abstract class in Java may have ✅ **zero or more** abstract methods.
7. Any method of an interface must be redefined in the Child class with access specifier ✅ **public**.
8. A class can extend ✅ **one** number of classes and ✅ **multiple** number of interfaces.

🌟 Explanation of Important Fill-Ups

- **Dynamic Binding:** Method overriding is resolved during runtime. Hence it is called 🚀 Dynamic Binding or 🚀 Runtime Polymorphism.
- **Mutable vs Immutable:** String objects cannot be modified directly. Hence String is ❌ Immutable. StringBuffer objects CAN be modified. Hence StringBuffer is ✅ Mutable.
- **Interface Method Visibility:** All interface methods are implicitly `public abstract`. Thus the overriding method must also remain `public`.

🎯 Conclusion

These fill-ups test core Java conceptual understanding involving inheritance, polymorphism, interfaces, abstraction, and String handling 🚀.

📄 QUESTION 2(d)

❓ **Create an abstract class Employee having private basic salary and abstract method getSalary().**
❓ **Inherit this class into Manager and Clerk.**
❓ **Managers get: 40% DA, 35% HRA.**
❓ **Clerks get: 20% DA, 15% HRA.**
❓ **Find total monthly salary disbursement for 5 managers and 15 clerks.**

✅ Answer

This question beautifully combines:

- 🚀 Abstract Classes
- 🧠 Inheritance
- 💡 Constructor Usage
- 🎯 Method Overriding
- 💻 Real-world Salary Processing

🌟 What is an Abstract Class?

An abstract class is a partially implemented class. It may contain:

- Abstract methods
- Concrete methods
- Constructors
- Variables

Abstract methods must be overridden in child classes.

✅ Complete Program

```
abstract class Employee {
    private double basicSal;

    Employee(double basicSal) {
        this.basicSal = basicSal;
    }

    double getBasicSal() {
        return basicSal;
    }

    abstract double getSalary();
}

class Manager extends Employee {
    Manager(double basicSal) {
        super(basicSal);
    }
```

```
    double getSalary() {
        double da = 0.40 * getBasicSal();
        double hra = 0.35 * getBasicSal();
        return getBasicSal() + da + hra;
    }
}

class Clerk extends Employee {
    Clerk(double basicSal) {
        super(basicSal);
    }

    double getSalary() {
        double da = 0.20 * getBasicSal();
        double hra = 0.15 * getBasicSal();
        return getBasicSal() + da + hra;
    }
}

class Demo {
    public static void main(String[] args) {
        Manager m = new Manager(50000);
        Clerk c = new Clerk(20000);

        double totalManagers = 5 * m.getSalary();
        double totalClerks = 15 * c.getSalary();

        double total = totalManagers + totalClerks;

        System.out.println("Total Salary = " + total);
    }
}
```

Output:

```
Total Salary = 787500.0
```

🌟 Important Concepts Used

- **Abstract Method:** `abstract double getSalary();` forces child classes to provide implementation.
- **Constructor Chaining:** `super(basicSal);` calls the parent constructor.
- **Runtime Polymorphism:** Different salary calculations occur depending on the object type.

🎯 Conclusion

Abstract classes provide common structure while child classes implement specialized behavior. This improves code reusability, scalability, and real-world object-oriented design 🚀.

📄 QUESTION 2(e)

❓ **How can you restrict a class of a package from being imported?**
❓ **Without importing a package, how can you inherit a particular class of a package?**
❓ **Discuss the advantage of static importing a package with suitable code.**
❓ **If two packages p1 and p2 both contain class A, what problem occurs? How can it be solved?**

✅ Restricting Package Import

A class can be restricted from importing by: 🚀 **Making the class non-public.** Only `public` classes are accessible outside packages.

Example:

```
package p1;
class Hidden {
    // This class cannot be imported outside package p1
}
```

✅ Inheriting Without Import

Use the fully qualified class name.

Example:

```
class Demo extends p1.A {
    // No import statement needed
}
```

🌟 Static Import

Static import allows direct access to static members without the class name prefix.

Example:

```
import static java.lang.Math.sqrt;

class Demo {
    public static void main(String[] args) {
        System.out.println(sqrt(25));
    }
}
```

Output:

```
5.0
```

🌟 Advantage of Static Import

- ✅ Cleaner code
- ✅ Better readability
- ✅ Reduces repeated class names

🌟 Problem with Same Class Name

Suppose both `p1.A` and `p2.A` are imported. Then ambiguity occurs ⚠️.

```
import p1.*;
import p2.*;

class Demo {
    A obj; // AMBIGUOUS: Compiler doesn't know if it's p1.A or p2.A
}
```

✅ Solution

Use fully qualified names to explicitly define which class you are referencing.

```
p1.A obj1;
p2.A obj2;
```

🎯 Conclusion

Packages improve modularity and organization in Java. Static imports simplify access to static members, while fully qualified names resolve package ambiguity issues 🚀.